

Kubernetes Network Policies

Since you're learning **KCNA as a beginner**, let's understand NetworkPolicy in the easiest way.

1. What is a Network Policy?

A NetworkPolicy controls which network traffic is allowed to or from Pods.

In very simple words:

 **NetworkPolicy = A security rule that controls Pod-to-Pod communication.**

Think of it like a **firewall for Pods**.

2. Why Do We Need Network Policies?

Imagine you have 3 Pods:

Frontend 

|



Backend 

|



Database 

Without network restrictions, Pods may be able to communicate more broadly than you want.

You might want:

Frontend → Backend 

Backend → Database 

Frontend → Database 

A NetworkPolicy helps you create these restrictions.

 **Easy memory:**

NetworkPolicy = Who can talk to whom?

3. Real-Life Example

Imagine an office with different rooms:

Reception

|



Office

|



Server Room

You don't want every person to enter the server room.

So you put a security rule:

Employees → Office 

Employees → Server Room 

Admin → Server Room 

NetworkPolicy does something similar for **network traffic between Pods**.

4. What Can a NetworkPolicy Control?

A NetworkPolicy can control:

Ingress 

Traffic **coming into a Pod**.

Pod A

|

| traffic



Pod B

For Pod B:

Pod A → Pod B



Ingress

Egress 

Traffic **going out from a Pod.**

Pod A



| traffic



Pod B

For Pod A:

Pod A → Pod B



Egress

Easy memory:

Ingress = IN 

Egress = OUT 

5. Ingress vs Egress

Type	Meaning	Example
------	---------	---------

Ingress	Incoming traffic	Backend receives traffic from Frontend
----------------	------------------	--

Egress	Outgoing traffic	Backend sends traffic to Database
---------------	------------------	-----------------------------------

Remember:

 **Ingress = Coming IN**

Egress = Going OUT

6. Example

Suppose we have:

Frontend Pod

Backend Pod

Database Pod

We want:

Frontend → Backend 

Backend → Database 

Frontend → Database 

We can create NetworkPolicies to enforce these communication rules.

7. NetworkPolicy Uses Pod Labels

NetworkPolicies commonly use **labels/selectors** to identify Pods.

For example:

podSelector:

matchLabels:

role: backend

This means:

"This policy applies to Pods having the label role=backend."

Example:

Pod

metadata:

labels:

role: backend

The policy can then control traffic involving those Pods.

8. Basic NetworkPolicy Example

Here's a simple example:

```
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
```

```
metadata:
```

```
  name: backend-policy
```

```
spec:
```

```
  podSelector:
```

```
    matchLabels:
```

```
      role: backend
```

```
  policyTypes:
```

```
    - Ingress
```

```
  ingress:
```

```
    - from:
```

```
      - podSelector:
```

```
        matchLabels:
```

```
          role: frontend
```

Let's break it down.

podSelector

```
podSelector:
```

matchLabels:

role: backend

→ Policy applies to **backend Pods**.

policyTypes

policyTypes:

- Ingress

→ We are controlling **incoming traffic**.

from

from:

- podSelector:

matchLabels:

role: frontend

→ Frontend Pods are allowed to send traffic to the selected backend Pods.

So:

Frontend 

|

| 

▼

Backend 

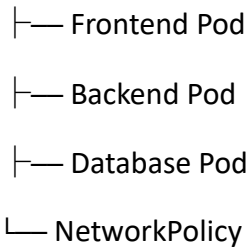
9. Important Concept: NetworkPolicy Is Namespace-Based

NetworkPolicies exist inside a **namespace**.

For example:

Namespace: production

|



A NetworkPolicy normally applies to Pods in its own namespace.

10. Does NetworkPolicy Automatically Block Everything?

This is an important beginner concept.

A Pod is generally **non-isolated by default**.

Once a NetworkPolicy selects a Pod for a particular direction, that direction becomes isolated and traffic must be allowed by applicable policies.

For example, if you create an ingress policy for a backend Pod:

Backend Pod



Ingress Policy



Only allowed sources can connect

So don't think:

"NetworkPolicy always blocks everything."

Instead:

It defines network isolation and allowed traffic for selected Pods.

11. NetworkPolicy Needs a Network Plugin

This is VERY important for KCNA.

Creating a NetworkPolicy object doesn't magically enforce the rules by itself.

Your Kubernetes networking implementation must support NetworkPolicy.

Examples of networking solutions that can provide NetworkPolicy enforcement include:

- Calico
- Cilium
- Other compatible network plugins

Easy memory:

 **NetworkPolicy = Rules**

 **Network plugin = Enforces the rules**

12. NetworkPolicy vs Service

Don't confuse these.

Service

A **Service** provides a stable way to access Pods.

Client



Service



Pods

NetworkPolicy

Controls whether network traffic is allowed.

Pod A



NetworkPolicy



Pod B

So:

Service = Provides access

NetworkPolicy = Controls access

13. NetworkPolicy vs RBAC

These are also different.

RBAC

Controls:

Who can perform Kubernetes API actions?

Example:

Developer



RBAC



Can get Pods

NetworkPolicy

Controls:

Which network traffic can reach Pods or leave Pods?

Example:

Frontend



NetworkPolicy



Backend

Easy memory:

RBAC



API permissions 

NetworkPolicy



Network traffic 

14. Real-World Example

Imagine an online shopping application:



Security requirement:

Internet → Frontend 

Frontend → Backend 

Backend → Database 

Frontend → Database 

NetworkPolicies can be used to restrict communication so that only the required paths are allowed.

This is a common **microservices security** use case.

15. Useful Commands

See NetworkPolicies

```
kubectl get networkpolicies
```

Short form:

```
kubectl get netpol
```

See policies in all namespaces

```
kubectl get networkpolicies -A
```

Describe a policy

```
kubectl describe networkpolicy <policy-name>
```

Example:

```
kubectl describe networkpolicy backend-policy
```

Get YAML

```
kubectl get networkpolicy backend-policy -o yaml
```

KCNA Revision Notes

NetworkPolicy

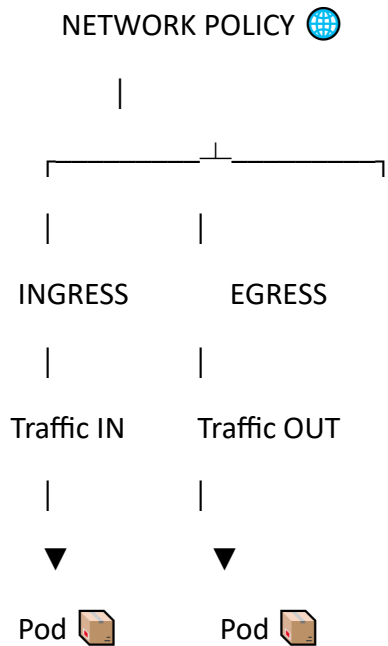
- Controls **network traffic to and from Pods**.
- Works like a **firewall/security rule for Pod networking**.
- Can control **Ingress** traffic.
- Can control **Egress** traffic.
- Uses **Pod selectors/labels** to identify Pods.
- NetworkPolicies are **namespace-scoped objects**.
- A NetworkPolicy requires a **network implementation that supports/enforces NetworkPolicy**.

Remember:

Ingress = IN 

Egress = OUT 

★ Super Easy Memory Trick



One-line exam definition:

A Kubernetes NetworkPolicy is an object used to control allowed network communication to and from selected Pods.